

Project Plan for Kibble

Anya Guyton, Geeoon Chung, Rohan Sabhaya, Elliot Norman, Giannah Donahoe

Table of Contents

1. Background	3
2. Objectives and Deliverables	4
2.1 Minimum Viable Product	4
2.2 Detailed Telemetry	4
2.3 Distributed Network	4
3. Milestones	5
3.1 Minimum Viable Product	5
3.1.1 Hardware Setup	5
3.1.2 Database Logging and Retrieval	5
3.1.3 ICMP Aliveness Check	5
3.1.4 Email Alert System	5
3.1.5 Overall System	5
3.1.6 Test Cases	5
3.1.7 Hardware Testing	5
3.2 Detailed Telemetry	6
3.2.1 Host Daemon and Central Communication	6
3.2.2 Daemon Telemetry Collection	6
3.2.3 Advanced Alert System	6
3.2.4 Overall System	6
3.2.5 Test Cases	6
3.2.6 Hardware Testing	6
3.3 Distributed Network	6
3.1.1 Hardware Setup	6
3.3.2 Decentralized Communication	6
3.3.3 Decentralized Logging	7
3.3.4 Decentralized Alerting	7
3.3.5 Data Visualization	7
3.3.6 Overall System	7
3.3.7 Test Cases	7
3.3.8 Hardware Testing	7
4. Work Breakdown Structure	8
5. Gantt Chart	9
6. Requirements	10
7. Concept Description and Sketch	11
8. Estimated Budget	16
9. Pugh Matrix	17

1. Background

Innoflight has tasked UW ECE ENGINE capstone students with building a test infrastructure monitoring system, named Kibble. The goal is to build a monitoring system for a test network. In the event that a test machine is not performing as expected, engineers should be notified. Notifications allow the problem to be fixed as soon as possible so the tests can resume. This is particularly useful for tests that are performed over long periods of time.

2. Objectives and Deliverables

2.1 Minimum Viable Product

This is the minimum product needed to fulfil the needs of the project. In our case, it implements all of the requirements listed in [6](#).

2.2 Detailed Telemetry

This deliverable will build on [2.1](#) to provide more specific monitoring information. Instead of relying on just latency and/or status of a device, it should monitor information about the device. (CPU usage, memory usage, temperature, etc.)

2.3 Distributed Network

This deliverable will build on [2.2](#) to allow the monitoring system to continue working in the event that a fault in the monitoring system occurs. This system will spread the monitoring infrastructure across the network, possibly even onto the devices themselves, or just with multiple monitoring devices on the network. The devices will communicate with each other to determine whether or not there are any faults in the testing infrastructure. This allows the system to continue if one or more of the monitoring devices goes down.

3. Milestones

3.1 Minimum Viable Product

3.1.1 Hardware Setup

Build the hardware setup with the Raspberry Pi test devices and the test PC connected together via a switch.

3.1.2 Database Logging and Retrieval

Set up the system for logging to the database. Also set up the system for retrieving entries from the database. This also involves determining the database schema.

3.1.3 ICMP Aliveness Check

Implement the aliveness check in Python using Scapy to send the ICMP echo request and receive the corresponding ICMP echo reply. Keep track of the latency as well.

3.1.4 Email Alert System

Implement the email alerting system. Could potentially set up an SMTP server, or choose a simpler alternative.

3.1.5 Overall System

Using the results of the previous milestones. Create a continuously running script that will check the status of devices on the network and alert in the event of an anomaly.

3.1.6 Test Cases

Design a series of test cases to verify the implementation of the requirements (i.e., traceability). Potentially use a library like the Robot Framework to automate the process.

3.1.7 Hardware Testing

Run the system on hardware with the test cases designed in [3.1.6](#).

3.2 Detailed Telemetry

3.2.1 Host Daemon and Central Communication

Design a protocol for a daemon running on the endpoint to communicate with the central test PC. Includes schema for data and communication type (raw TCP/UDP or application level such as REST).

3.2.2 Daemon Telemetry Collection

Design a system for collecting telemetry data about the endpoint (CPU usage, memory usage, disk space, temperature, traffic in/out). Ideally, it is OS independent, but this may be difficult.

3.2.3 Advanced Alert System

Design a system that alerts when an endpoint is behaving abnormally. It does not need to be down for an alert to occur.

3.2.4 Overall System

Using the results of the previous milestones. Create a continuously running script that will check the status of devices on the network and alert in the event of an anomaly.

3.2.5 Test Cases

Design a series of test cases to verify the implementation of the requirements (i.e., traceability). Potentially use a library like the Robot Framework to automate the process.

3.2.6 Hardware Testing

Run the system on hardware with the test cases designed in [3.2.5](#).

3.3 Distributed Network

3.1.1 Hardware Setup

Build the hardware setup with the Raspberry Pi test devices and multiple test PCs connected together via switches/routers. Requires multiple subnets and routing.

3.3.2 Decentralized Communication

Design a system that allows for multiple test PCs to be deployed at different parts of the network while still communicating with each other, possibly across subnets.

3.3.3 Decentralized Logging

Design a system that prevents redundant logging from multiple test PCs.

3.3.4 Decentralized Alerting

Design a system that prevents redundant alerting from multiple test PCs.

3.3.5 Data Visualization

Design a system that allows an engineer to visually see the network topology and which device is behaving abnormally.

3.3.6 Overall System

Using the results of the previous milestones. Create a continuously running script that will check the status of devices on the network and alert in the event of an anomaly.

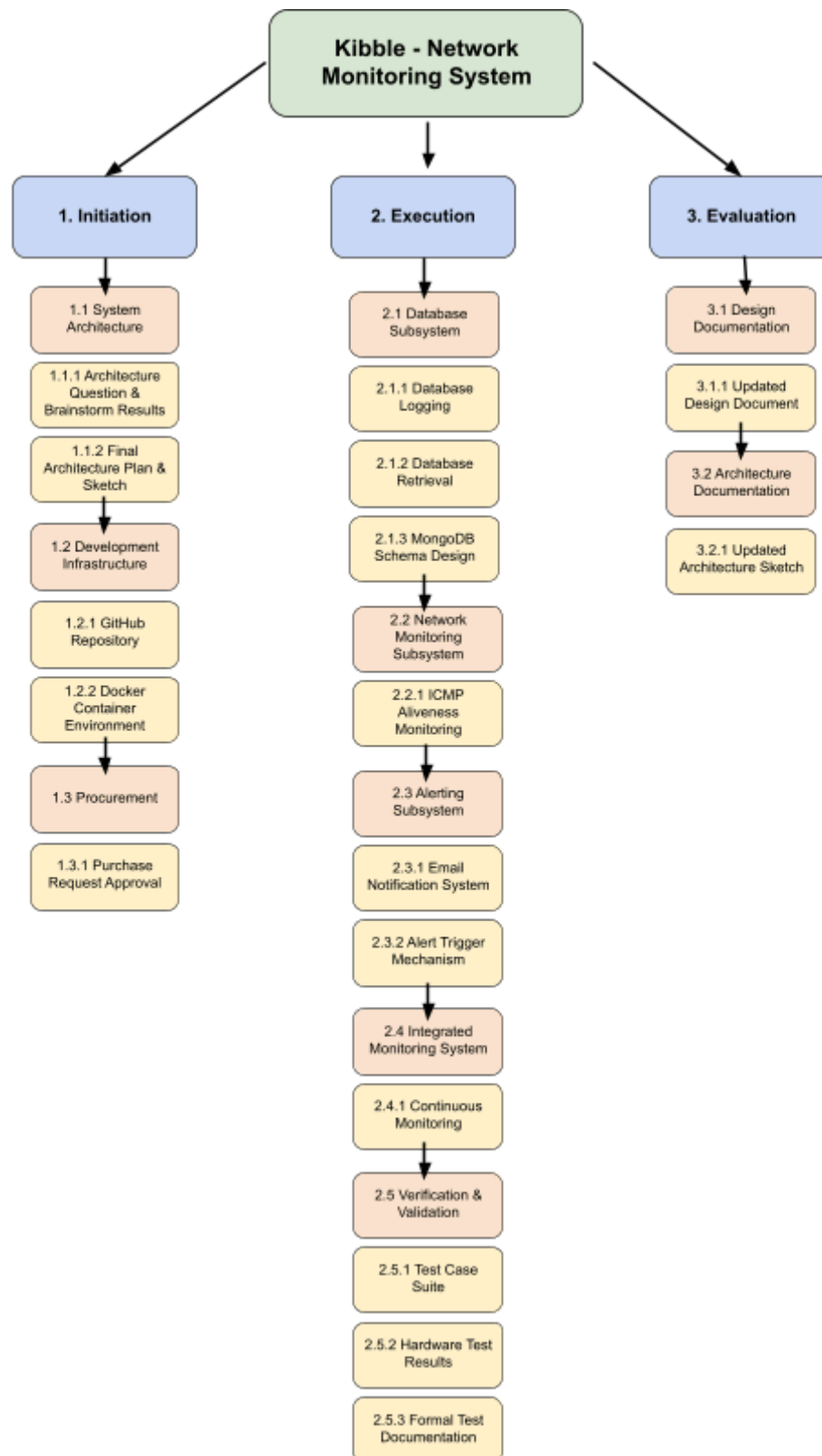
3.3.7 Test Cases

Design a series of test cases to verify the implementation of the requirements (i.e., traceability). Potentially use a library like the Robot Framework to automate the process.

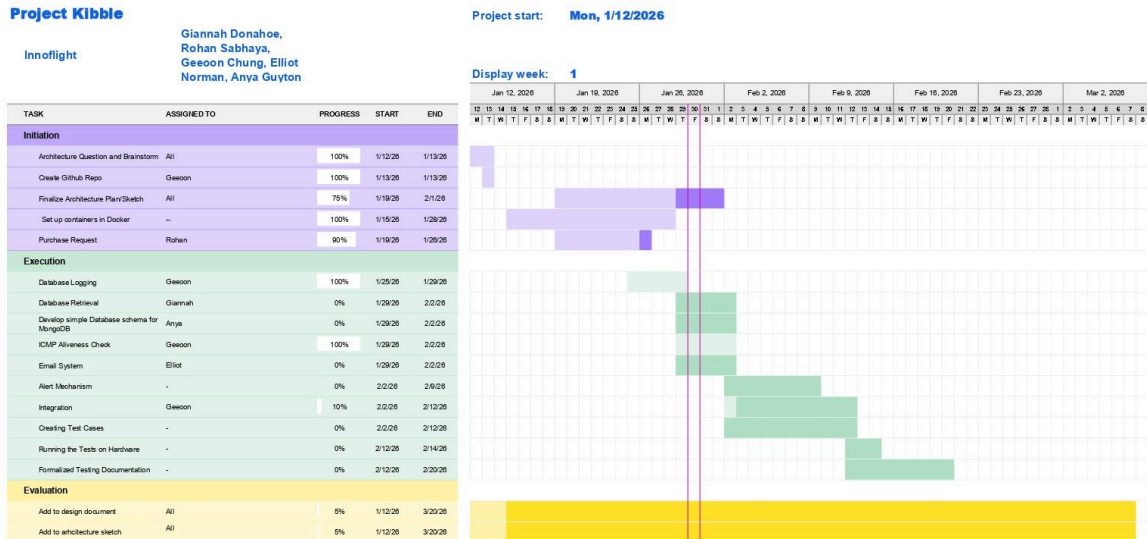
3.3.8 Hardware Testing

Run the system on hardware with the test cases designed in [3.3.7](#).

4. Work Breakdown Structure



45. Gantt Chart



Link-

https://uwnetid-my.sharepoint.com/:x/r/personal/anyaguyt_uw_edu/layouts/15/Doc.aspx?sourcedoc=%7B870F30F9-B5E0-439C-9648-3FFA0DD1298C%7D&file=Gantt%20Chart.xlsx&action=default&mobileredirect=true

6. Requirements

The following requirements have been provided by Innoflight.

1. Monitoring Capability
 - 1.1. The system should continuously monitor the operational status of all connected test equipment. This includes items like power status, latency/responsiveness, and communication availability.
2. Fault Detection and Alerting
 - 2.1. The system should automatically detect fault conditions with any monitored parameters. Loss of connectivity, increased latency, etc.
 - 2.2. Upon detection of a fault condition, the system should generate an alert and notify designated personnel through one or more configurable alert mechanisms (e.g., email, SMS, MS Teams).
 - 2.3. Alerts should include sufficient diagnostic information to identify the affected device and fault type.
3. Reliability and Availability
 - 3.1. The system should remain operational for the full duration of long-running tests without requiring system restarts.
 - 3.2. The system should continue monitoring unaffected devices if a single device failure occurs.
4. Configuration and Scalability
 - 4.1. The system should support monitoring of multiple test devices simultaneously.
 - 4.2. The system should support running multiple instances (multiple monitors) simultaneously.
5. Data Logging and Traceability
 - 5.1. The system should support logging monitoring data and fault events with timestamps to a centralized source.

7. Concept Description and Sketch

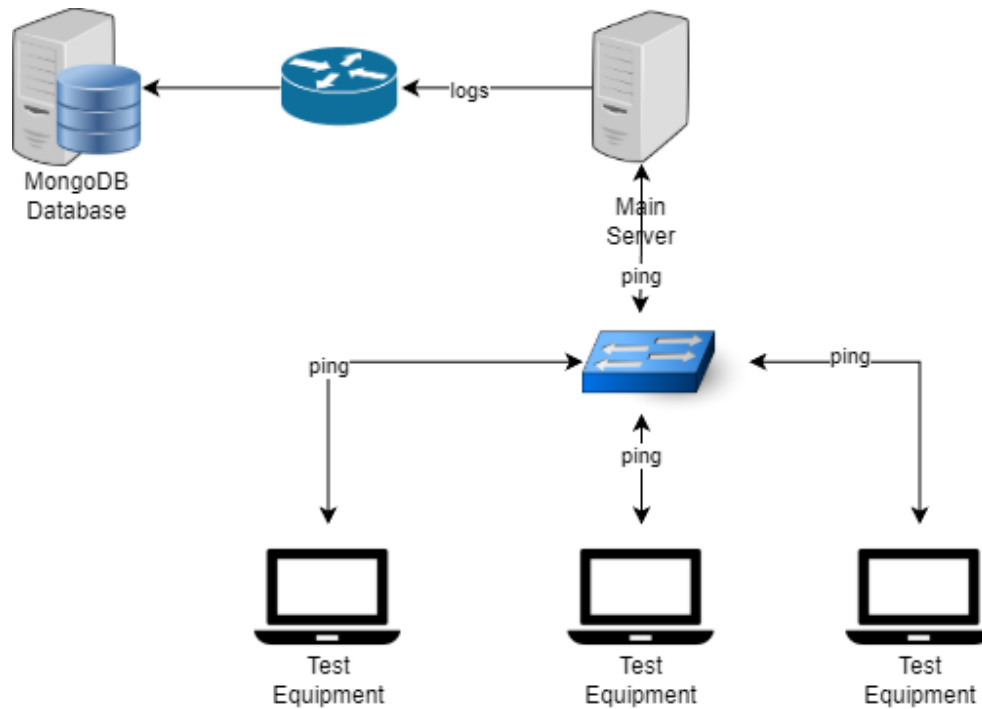


Figure 1: Architectural Diagram

At a high level, the system should work in a network with a centralized server. This server will ping the test equipment to check for their status. Once this is finished, any abnormal results are stored onto the MongoDB server.

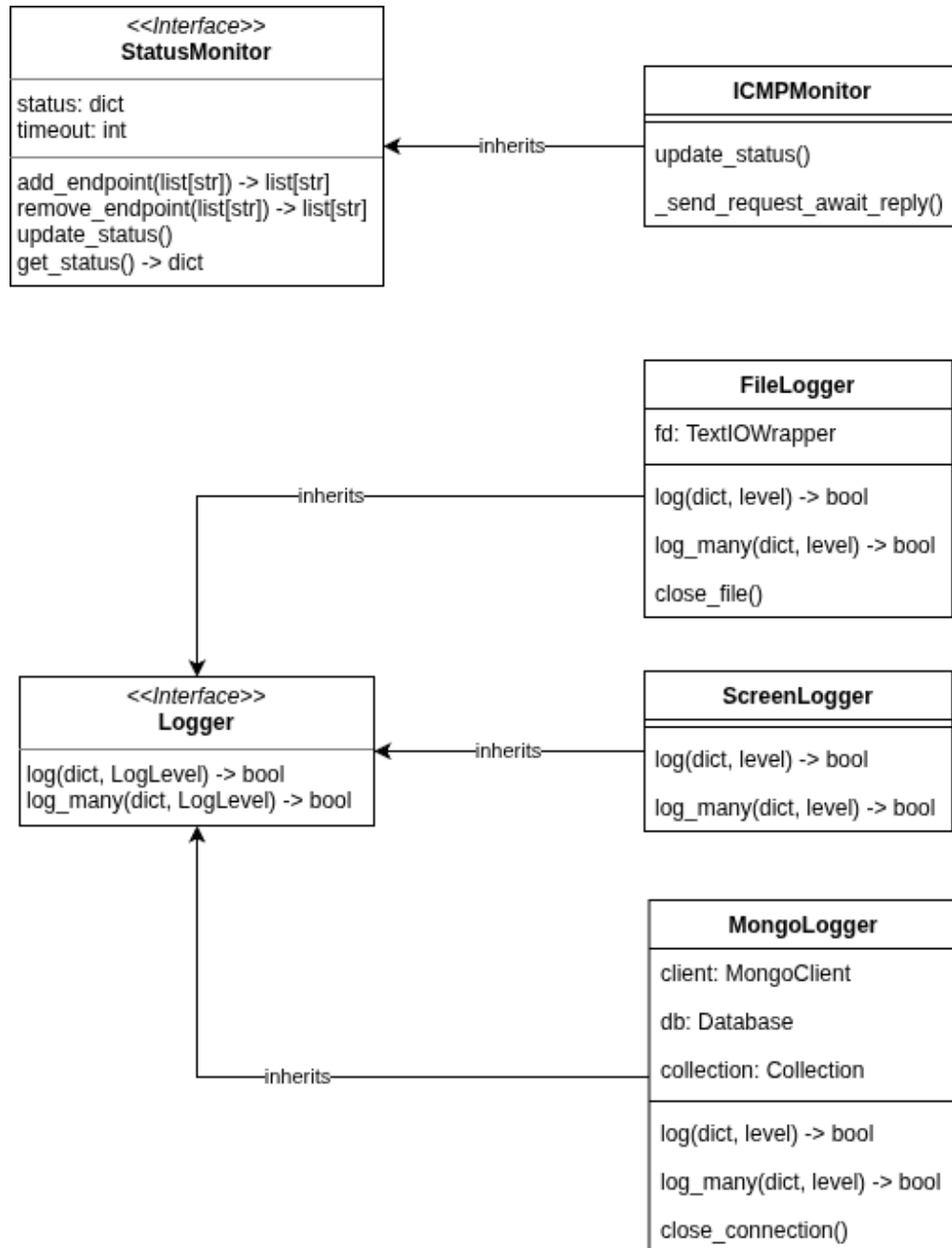


Figure 2: Inheritance Structure

The software monitoring the test equipment is designed to be as extensible as possible. The pinging of the test equipment is done via an **ICMPMonitor** class which inherits from a generic **StatusMonitor** class, which provides an interface for the main program to interact with potentially many different types of status retrieval methods. The logging of events is also achieved through a similar inheritance structure. A base **Logger** class provides an interface to interact with any of the three derived classes, **ScreenLogger**, **FileLogger**, and **MongoLogger**.

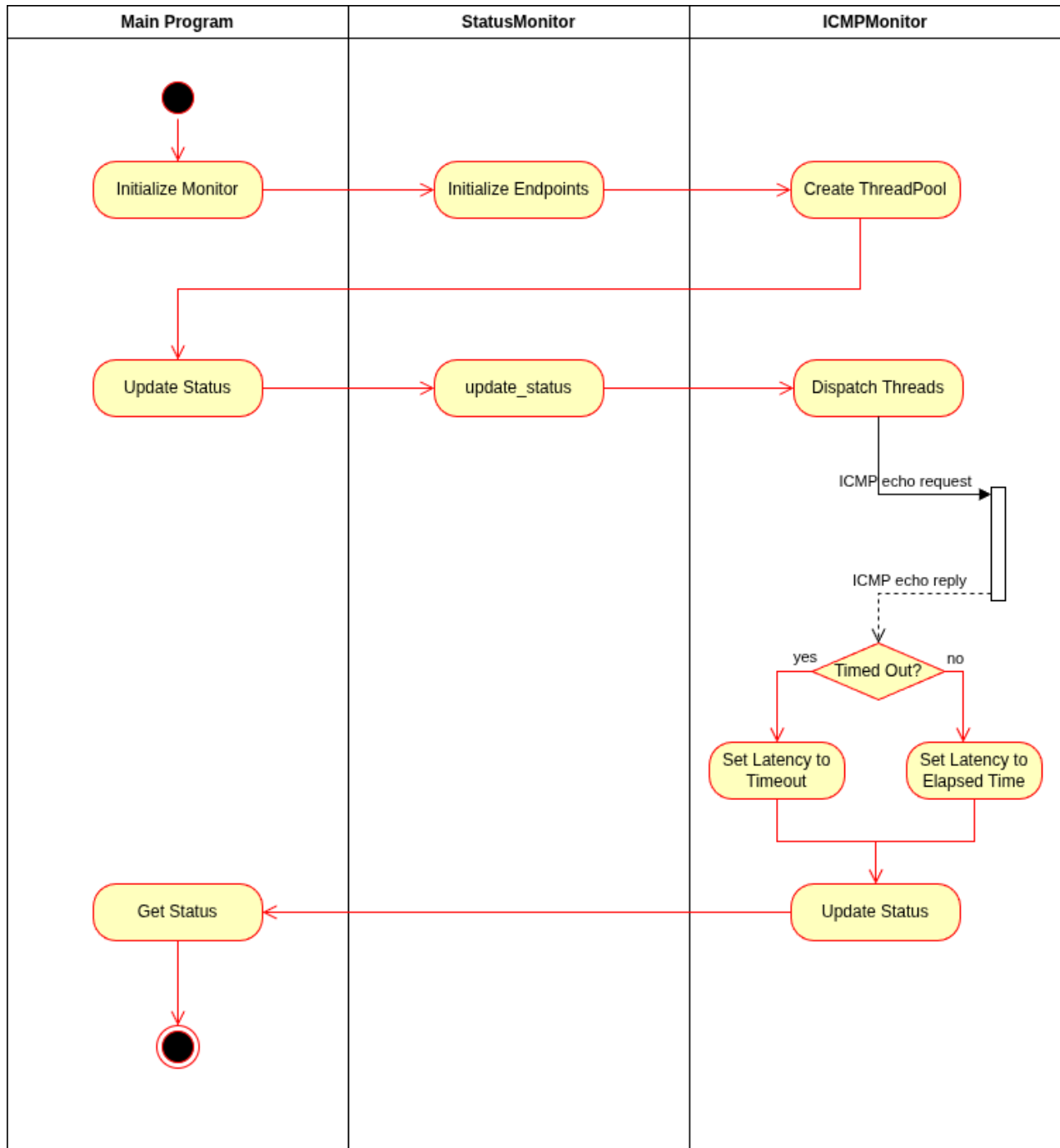


Figure 3: StatusMonitor Process with ICMPMonitor

The ICMPMonitor concurrently checks multiple endpoints through the use of threading to quickly determine the status of all endpoints.

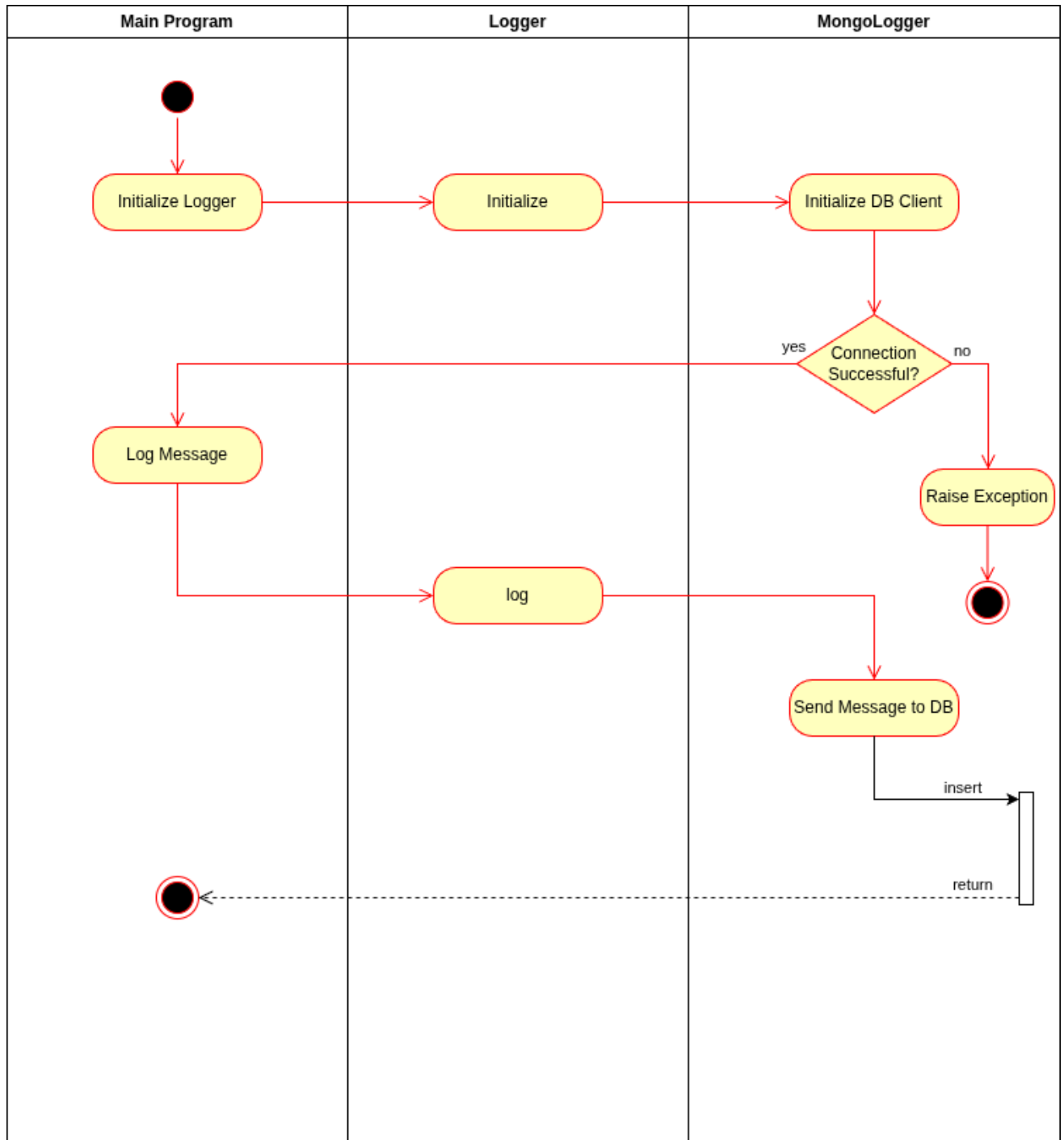


Figure 4: Logger Process with MongoLogger

The MongoLogger connects to the MongoDB database and logs any messages.

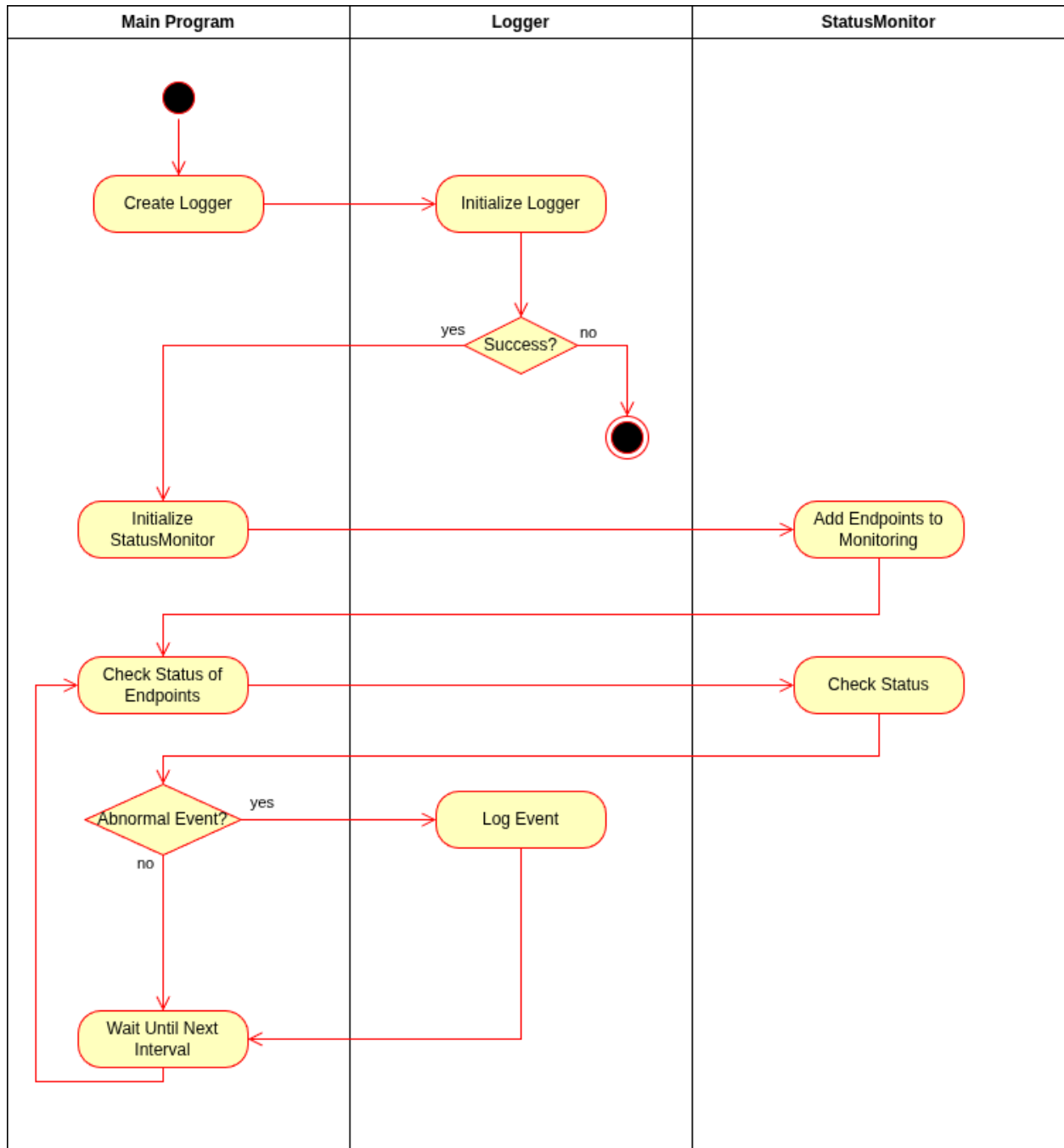


Figure 5: Overall Process

The main program will initialize a logger and a monitor, then continuously check the status of the endpoints at regular intervals, logging abnormal events as they occur.

8. Estimated Budget

The budget allocated for the Kibble Project from Innoflight and the University of Washington is \$1000.

As of now, this budget is very satisfactory with the project plan, and should be more than enough to achieve the minimum viable product. The only current usage of the budget has been to purchase a raspberry pi, a router, a network switch and two ethernet cables for a grand total of around ~\$105.

There are no other plans to use the remaining budget of approximately \$895 on any additional supplies to accomplish the current project scope.

9. Pugh Matrix

Project approaches:

Option A (Baseline) : MVP, Only ping/aliveness checks, Logs downtime, Simple email alerting

Option B: Latency-Aware Connectivity- logs downtime and slow responses

Option C: Detailed Connection Tests- pinging, latency, etc, more aliveness checks

Option D (Full Stretch): Additionally aliveness checks (Like HTTP), distributed monitoring, event severity determination

+1 : Better than baseline

0 : Same as baseline

-1 : Worse than baseline

Criteria	Weight	A (Baseline)	B	C	D
Ease of Implementation	3	0	-1	-1	-1
Reliability	4	0	+1	+1	+1
Monitoring Coverage	4	0	+1	+1	+1
Scalability	3	0	0	0	+1
Cost	3	0	0	0	-1 ?
Learning Opportunity	2	0	+1	+1	+1
Maintability/ Extendability	2	0	0	0	-1
Usefulness	4	0	0	+1	+1
Total		0	+7	+11	+9

Therefore, the MVP is a good start, and we should plan our goals and deadlines for now to attempt to finish C for the best interest of our learning experience and Innoflight. Option D can be considered our “stretch” should we find we have extra time and resources later on.